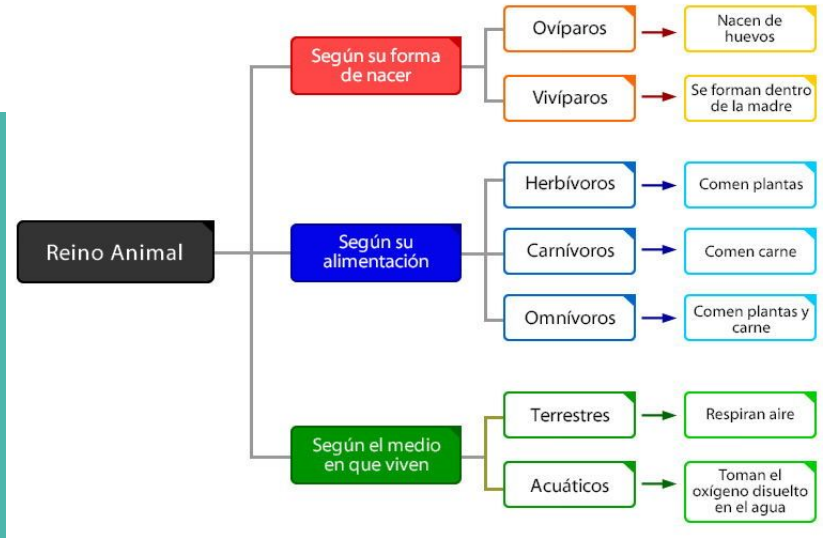


Estrategias para probar (Clasificación de pruebas)



Estrategias para probar (Clasificación de pruebas)

1

Según la Accesibilidad

2

Según la Ejecución

3

Según el Método

4

Según el Alcance

5

Según el Nivel

Estrategias para probar (Clasificación de pruebas)

1

Según la Accesibilidad

2

Según la Ejecución

3

Según el Método

4

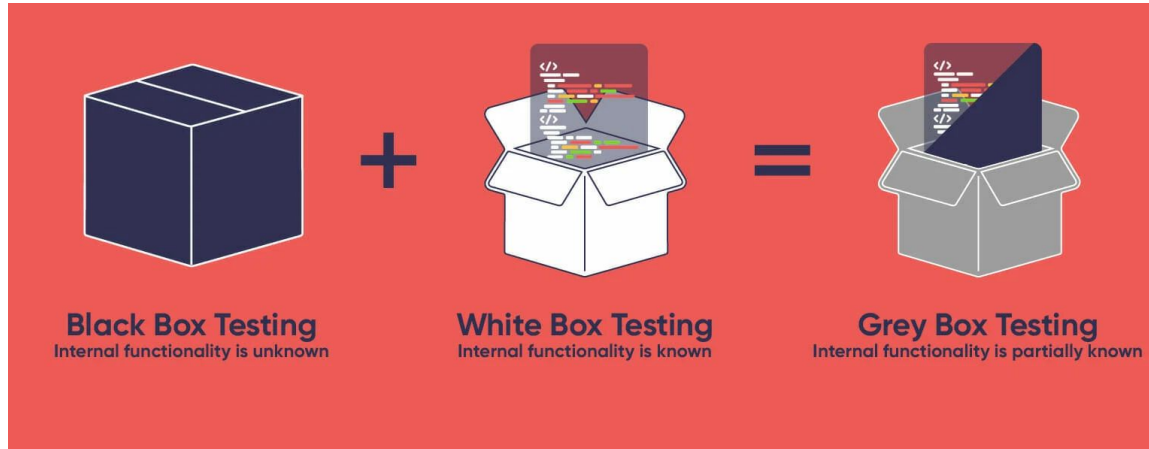
Según el Alcance

5

Según el Nivel



1- Según la Accesibilidad



1- Según la Accesibilidad - Black Box Testing

Black Box Testing (Test de Caja Negra) es un método de prueba de software en el que se prueban las funcionalidades de las aplicaciones de software sin tener conocimiento de la estructura interna del código, los detalles de implementación y las rutas internas. Las pruebas de caja negra se centran principalmente en la entrada y salida de aplicaciones de software y se basan completamente en los requisitos y especificaciones de software. También se conoce como pruebas de comportamiento.



1- Según la Accesibilidad - Black Box Testing - Pro & Cons

Ventajas

- No es necesario que el probador tenga conocimientos técnicos. Es importante probar poniéndose en el lugar del usuario y pensar desde el punto de vista del usuario.
- Las pruebas pueden comenzar una vez que se realiza el desarrollo del proyecto/aplicación. Tanto los probadores como los desarrolladores trabajan de forma independiente sin interferir en el espacio de los demás.
- Es más eficaz para aplicaciones grandes y complejas.
- Los defectos e inconsistencias se pueden identificar en las primeras etapas de la prueba.

Desventajas

- Sin ningún conocimiento técnico o de programación, hay posibilidades de ignorar las posibles condiciones del escenario a probar.
- La cobertura completa de la prueba no es posible para proyectos grandes y complejos.

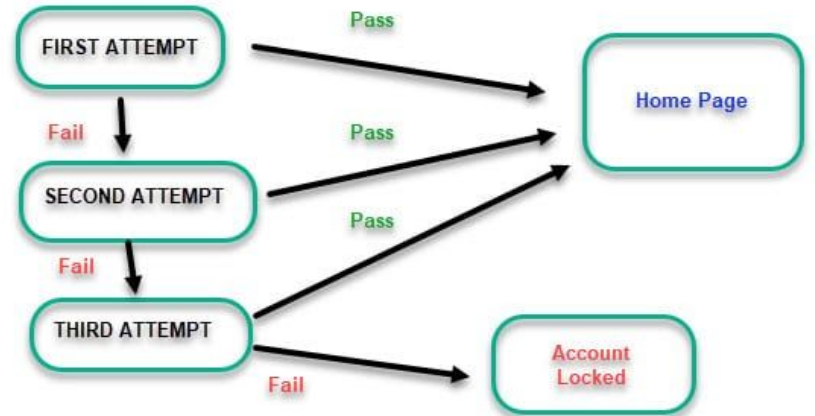
Black Box Testing - Ejemplo

AGE * Accepts value from 18 to 60

Equivalence Class Partitioning

Invalid	Valid	Invalid
≤ 17	18-60	≥ 61

STATE TRANSITION TESTING



1- Según la Accesibilidad - White Box Testing

En las pruebas de caja blanca se evalúa la estructura interna, el diseño y la codificación del software para verificar el flujo de entrada-salida y mejorar el diseño, la usabilidad y la seguridad.

En las pruebas de caja blanca, **el código es visible para los evaluadores**, por lo que también se denomina prueba de caja transparente, prueba de caja abierta, prueba de caja transparente, prueba basada en código y prueba de caja de cristal.



White Box Testing - ¿Qué se verifica?

Las pruebas de caja blanca implican la prueba del código de software para lo siguiente:

- Brechas de seguridad interna
- Rutas rotas o mal estructuradas en los procesos de codificación
- El flujo de entradas específicas a través del código
- Resultado esperado
- La funcionalidad de los bucles condicionales
- Comprobación de cada instrucción, objeto y función de forma individual

White Box Testing - Pro & Cons

Ventajas

- Optimización del código mediante la búsqueda de errores ocultos.
- Los casos de prueba de caja blanca se pueden automatizar fácilmente.
- Las pruebas son más exhaustivas, ya que generalmente se cubren todas las rutas de código.
- Las pruebas pueden comenzar temprano en *SDLC* incluso si la *GUI* no está disponible.

Desventajas

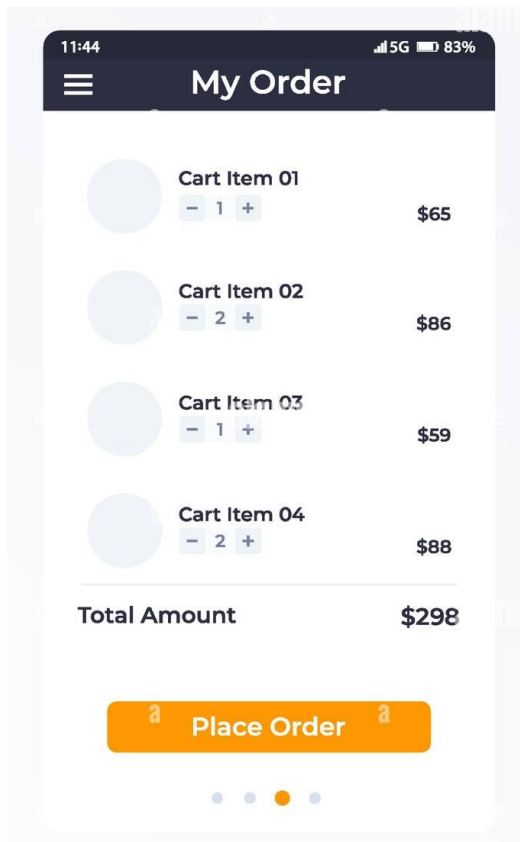
- Las pruebas de caja blanca pueden ser bastante complejas y costosas.
- Las pruebas de caja blanca realizadas por los desarrolladores usualmente no son detalladas y pueden dar lugar a errores de producción.
- Las pruebas de caja blanca requieren recursos profesionales con una comprensión detallada de la programación y la implementación.
- Las pruebas de caja blanca normalmente requieren más tiempo.

1- Según la Accesibilidad - Grey Box Testing

Las pruebas de caja gris son una técnica de prueba de software para probar un producto o aplicación de software con **conocimiento parcial de la estructura interna de la aplicación**. El propósito de las pruebas de caja gris es buscar e identificar los defectos debidos a una estructura de código incorrecta o al uso incorrecto de las aplicaciones.



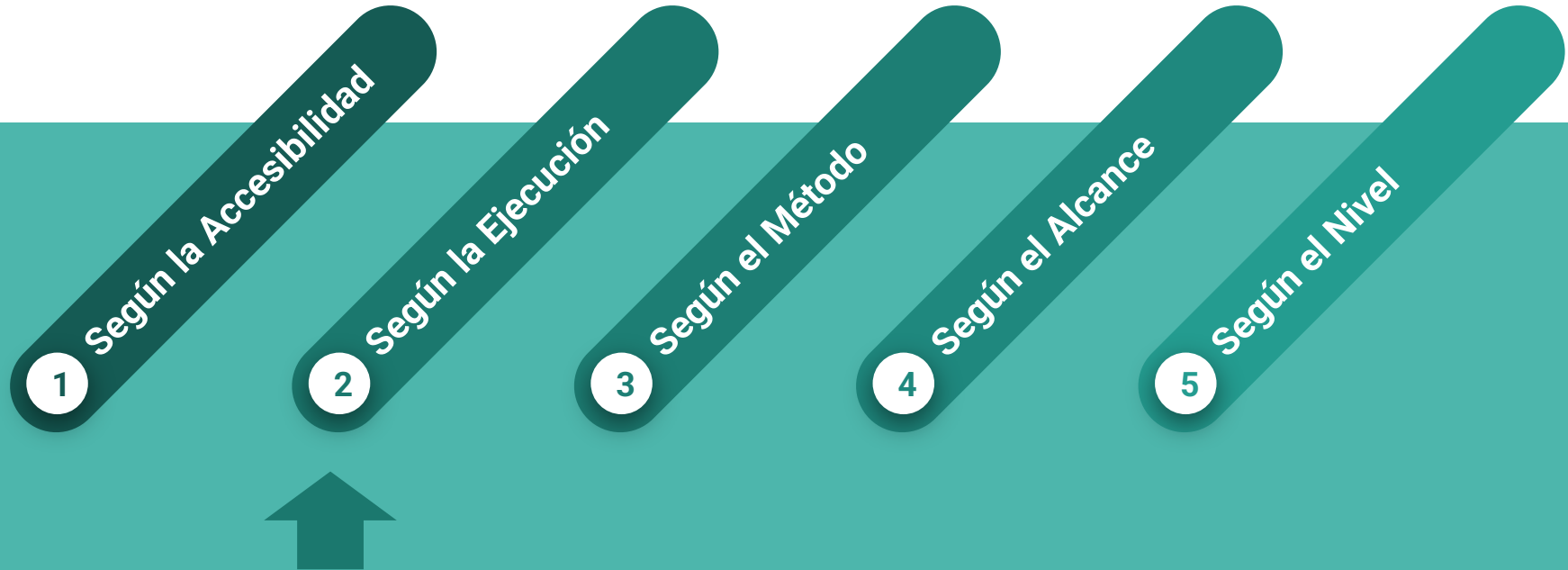
Grey Box Testing - Ejemplo



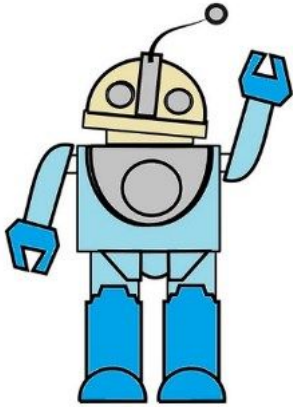
```
-- Consulta para recuperar una orden por su número de orden (order_id)
SELECT *
FROM ordenes
WHERE order_amount = 298; -- Reemplaza 298 con el monto a consultar
```



Estrategias para probar (Clasificación de pruebas)



2- Según la Ejecución



**Pruebas
Automatizadas**



Pruebas Manuales



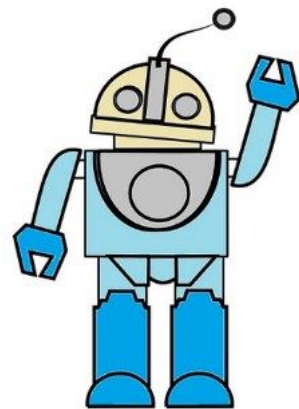
**Pruebas
semiautomáticas**

2- Según la Ejecución - Pruebas Automatizadas

Las pruebas automatizadas son una técnica de prueba de software que se realiza mediante el uso de herramientas especiales de software de pruebas automatizadas para ejecutar un conjunto de casos de prueba.

Las pruebas automatizadas son la aplicación de herramientas de software para automatizar un proceso manual impulsado por humanos para revisar y validar un producto de software. La mayoría de los proyectos modernos de software ágil y *DevOps* ahora incluyen pruebas automatizadas desde el inicio.

Sin embargo, para apreciar plenamente el valor de las pruebas automatizadas, es útil comprender cómo era la vida antes de que se adoptaran ampliamente.



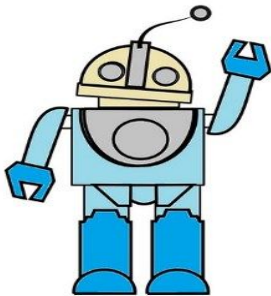
2- Según la Ejecución - Pruebas Manuales

Las pruebas manuales son un tipo de pruebas de software en las que los casos de prueba son ejecutados manualmente por un evaluador sin utilizar ninguna herramienta automatizada. El propósito de las pruebas manuales es identificar los errores, problemas y defectos en la aplicación de software. Las pruebas manuales de software son la técnica más primitiva de todos los tipos de pruebas y ayudan a encontrar errores críticos en la aplicación de software.

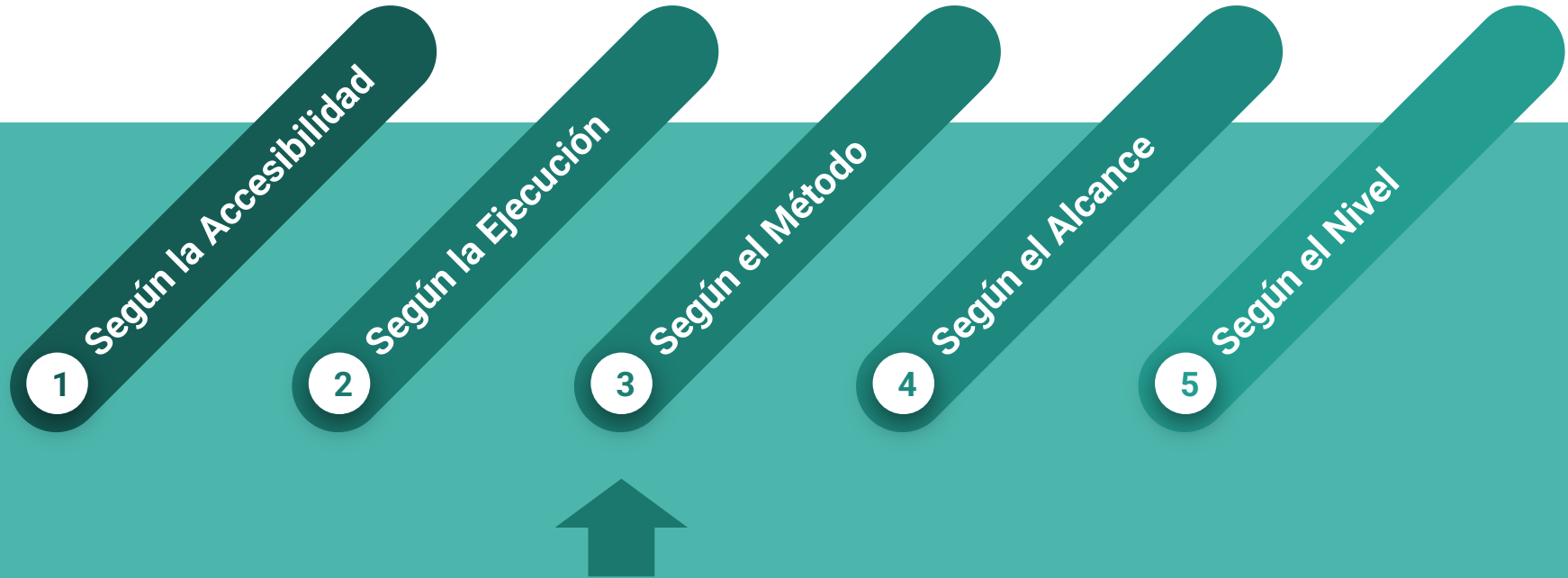


2- Según la Ejecución - Pruebas Semiautomáticas

Las pruebas semiautomáticas poseen pasos automáticos pero también exigen alguna interacción manual. Por ejemplo, para resolver un *captcha* previo que autoriza el inicio efectivo del uso del sistema. También puede ocurrir pues se quiere automatizar sólo parte de las pruebas pues otra parte probablemente aún debe cambiar bastante (por ej. *GUI* que habilita precondiciones de la prueba deseada). Existen técnicas y herramientas para facilitar la automatización abstrayéndose de las partes (aún) no automatizadas. Pero dependiendo del sistema bajo pruebas y de las pruebas por realizarse, ocasionalmente conviene combinar la automatización con algunos pasos manuales.



Estrategias para probar (Clasificación de pruebas)



3 - Según el Método



Pruebas Estáticas



Pruebas Dinámicas

3 - Según el Método - Pruebas Estáticas

La pruebas estáticas no requieren de la ejecución de software para ser realizadas. Parte del objetivo de las pruebas estáticas es la revisión de productos de trabajo como documentos de requerimientos, casos de prueba, planes de prueba, código, guías de usuario.

Estas **pruebas se enfocan en la prevención de defectos y en la detección temprana** de los mismos, ya que se pueden realizar en cualquier etapa del ciclo de vida de software según la información que se tenga disponible.

Se utilizan distintas ejecuciones:

- Revisión manual
- Revisión automática del código

¿Qué se puede detectar?

Detección de vulnerabilidades, código duplicado, código muerto, estándares de codificación, complejidad ciclomática, etc.



<https://medium.com/@elliott.chiu/pylint-integrate-to-vscode-5e56dcface31>

3 - Según el Método - Pruebas Dinámicas

Las pruebas dinámicas son aquellas que **se realizan mientras el código está en ejecución**. Tienen como objetivo asegurar que el software se comporte de acuerdo con los requerimientos del negocio mediante la realización de pruebas funcionales y no funcionales.

Estas pruebas se enfocan en la detección y confirmación de la corrección de defectos en el software. Por lo general se realizan en una etapa más tardía que las pruebas estáticas, por lo cual, los defectos encontrados en estas son más costosos.



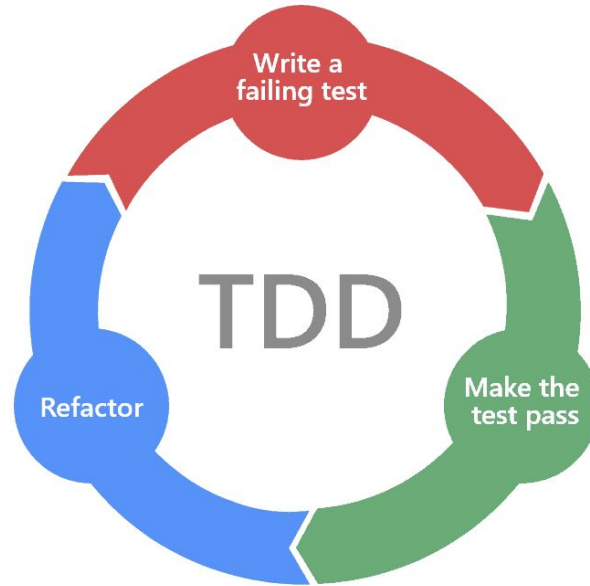
¡A trabajar!

Actividad - Análisis según Accesibilidad y Ejecución

Analice la planificación de las pruebas que realizó en la clase pasada y explique que tendría que ocurrir para que cada caso sea probado en los diferentes niveles de accesibilidad y ejecución.

Automatización de Pruebas & Refactorización

Test Driven Development (Desarrollo Guiado por Pruebas)



Actividad práctica

Ver video [Test Driven Development Tutorial For Beginners \(youtube.com\)](https://www.youtube.com/watch?v=8mX8UgWw8e0)

Practicar hasta implementar la solución a al menos 2 problemas, el FizzBuzz más otro de su elección.

Agregar el código de su práctica <https://cyber-dojo.org/> al *readme* de su github

Plazo hasta el miércoles 08/05